

Unit 4: Natural Language Processing & Pattern Recognition

Complete Notes for 5-Mark Questions

1. Understanding Natural Languages

Natural Language Processing (NLP) is a subfield of Artificial Intelligence concerned with enabling computers to understand, interpret, and generate human language in a meaningful way.

Key Characteristics of Natural Language:

- **Ambiguity:** Words and sentences can have multiple meanings (lexical, syntactic, semantic ambiguity).
- **Context Dependency:** Meaning changes based on context, background knowledge, and conversation history.
- **Irregularity:** Unlike formal languages, natural languages have many exceptions to rules.
- **Creativity:** Humans can produce and understand infinitely many new sentences.
- **Vagueness:** Natural language expressions are often imprecise.

Levels of NLP Analysis:

1. **Phonological Analysis** – Study of sounds and phonemes.
2. **Morphological Analysis** – Study of word structure (roots, prefixes, suffixes).
3. **Lexical Analysis** – Identifying and analyzing words (tokenization, POS tagging).
4. **Syntactic Analysis (Parsing)** – Analyzing grammatical structure of sentences.
5. **Semantic Analysis** – Extracting meaning from sentences.
6. **Pragmatic Analysis** – Understanding intent and context of language use.
7. **Discourse Analysis** – Understanding multi-sentence structure and coherence.

Challenges in NLP:

- Word sense disambiguation
 - Coreference resolution
 - Named entity recognition
 - Figurative language (idioms, metaphors)
-

2. Various Approaches of NLP

A. Rule-Based Approach

- Uses manually crafted linguistic rules (grammar rules, dictionaries).
- **Advantages:** Precise, interpretable, no need for training data.
- **Disadvantages:** Expensive to build, doesn't scale, brittle to new inputs.
- Example: ELIZA chatbot, early MT systems.

B. Statistical Approach

- Uses probabilistic models trained on large text corpora.
- Key techniques: n-gram models, Hidden Markov Models (HMM), Bayesian methods.
- **Advantages:** Handles ambiguity, scalable.
- **Disadvantages:** Requires large data, less interpretable.

C. Machine Learning Approach

- Uses algorithms like SVM, Naive Bayes, Decision Trees trained on labeled data.
- Applications: Text classification, sentiment analysis, NER.

D. Deep Learning Approach

- Uses neural networks (RNN, LSTM, Transformer).
- **BERT, GPT:** Pre-trained language models fine-tuned for tasks.
- **Advantages:** State-of-the-art performance, minimal feature engineering.
- **Disadvantages:** Requires massive data and computation.

E. Hybrid Approach

- Combines rule-based and statistical/ML methods for better accuracy.
-

3. Parsing Techniques

Parsing is the process of analyzing a string of symbols (sentence) according to the rules of a formal grammar to determine its grammatical structure.

A. Top-Down Parsing

- Starts from the start symbol (S) and tries to derive the input sentence.
- Expands rules until terminals match input.
- **Problem:** Left-recursion causes infinite loops; inefficient due to backtracking.
- Example: Recursive Descent Parser.

B. Bottom-Up Parsing

- Starts from the input words and reduces them using grammar rules until the start symbol is reached.
- Example: Shift-Reduce Parser, CYK Algorithm.
- **CYK (Cocke-Younger-Kasami)**: Dynamic programming-based; works with CNF (Chomsky Normal Form) grammars. Time complexity: $O(n^3)$.

C. Chart Parsing (Earley Parser)

- Combines top-down prediction with bottom-up completion.
- Efficient; handles ambiguous grammars.
- Uses a chart (table) to store partial parses and avoid redundant computation.
- Time complexity: $O(n^3)$ in worst case, $O(n)$ for unambiguous grammars.

D. Left-Corner Parsing

- A hybrid of top-down and bottom-up.
- Avoids left-recursion problem of top-down.

E. Head-Driven Parsing

- Uses the head of a phrase to guide parsing decisions.
-

4. Context-Free Grammars (CFG)

A Context-Free Grammar is defined by a 4-tuple: $G = (V, T, P, S)$ where:

- V = Set of non-terminal symbols (variables)
- T = Set of terminal symbols (words)
- P = Set of production rules of the form $A \rightarrow \alpha$ ($A \in V, \alpha \in (V \cup T)^*$)
- S = Start symbol

Example CFG for English:

```
S → NP VP
NP → Det N | N
VP → V NP | V
Det → "the" | "a"
N → "dog" | "cat"
V → "sees" | "chases"
```

Properties:

- A non-terminal can be replaced regardless of surrounding context.

- Can represent recursive structures (nested clauses).
- Cannot capture all natural language phenomena (e.g., agreement, movement).

Parse Tree: A tree showing how a sentence is derived from the grammar.

Ambiguity in CFG: A grammar is ambiguous if a sentence has more than one parse tree.

- Example: "I saw the man with the telescope" — two interpretations.

5. Transformational Grammars

Introduced by **Noam Chomsky (1957)** in *Syntactic Structures*.

A Transformational Grammar (TG) distinguishes between:

- **Deep Structure:** Underlying abstract syntactic form (reflects meaning).
- **Surface Structure:** The actual spoken/written form.

Transformations convert deep structures to surface structures via rules:

- **Passivization:** "The dog bit the man" → "The man was bitten by the dog"
- **Question Formation:** "She is leaving" → "Is she leaving?"
- **Negation:** "He eats" → "He does not eat"
- **Relativization, Topicalization, etc.**

Components:

1. **Phrase Structure Rules** - Generate deep structure (like CFG).
2. **Transformational Rules** - Map deep to surface structure.
3. **Morphophonological Rules** - Handle pronunciation changes.

Chomsky Hierarchy (formal language classes):

Type	Grammar	Automaton
0	Unrestricted	Turing Machine
1	Context-Sensitive	Linear Bounded Automaton
2	Context-Free	Pushdown Automaton
3	Regular	Finite Automaton

6. Transition Nets (Finite State Automata in NLP)

A **Transition Network** (also called Recursive Transition Network or RTN) is a graph-based formalism for recognizing language patterns.

Structure:

- **Nodes:** States (one initial state, one or more final states).
- **Arcs:** Transitions labeled with words, word categories, or non-terminal labels.

Simple Transition Net:

- Nodes represent stages in parsing.
- Arcs are labeled with expected words or categories.
- **Parsing:** Follow arcs matching input words; if final state is reached, sentence is accepted.

Example: Recognizing "a dog sees a cat":

START → (Det) → (N) → (V) → (Det) → (N) → END

Limitations of Simple Transition Nets:

- Cannot handle recursion well (e.g., nested clauses).
- Equivalent to Regular Grammars (less powerful than CFG).

Recursive Transition Networks (RTN):

- Arcs can be labeled with names of other networks (non-terminals).
- Allows recursion: a network can call itself.
- Equivalent in power to CFG.

7. Augmented Transition Networks (ATN)

Developed by **William Woods (1970)**, ATNs extend RTNs with:

- **Registers:** Memory slots to store information during parsing (subject, verb, object, etc.).
- **Conditions on arcs:** Arcs can have conditions (e.g., number agreement check).
- **Actions on arcs:** Arcs can perform operations (e.g., store word in a register, build a structure).

Why ATN is Powerful:

- Can handle complex linguistic phenomena: agreement, movement (wh-questions), passivization.
- Can build semantic representations during parsing.

- More expressive than CFG.

ATN Parsing Example (Wh-question):

- "What did the cat eat?" — ATN can use a HOLD register to store "what" and fill it into the object slot after verb processing.

Arc Types in ATN:

Arc Label	Meaning
WORD x	Match literal word x
CAT c	Match word of category c
PUSH NP	Invoke NP sub-network
JUMP	Transition without consuming input
POP	Return from sub-network with built structure

Limitations:

- Procedural and hard to maintain.
- Non-declarative (difficult to separate linguistic knowledge from control).

8. Fillmore's Case Grammar

Proposed by **Charles Fillmore (1968)** in "The Case for Case."

Core Idea: The meaning of a sentence is best understood in terms of the semantic roles (cases) that noun phrases play relative to the verb.

Deep Cases (Semantic Roles):

Case	Description	Example
Agent	Instigator of action	<i>John</i> opened the door
Patient/Object	Entity affected	John opened <i>the door</i>
Instrument	Means used	John opened it with <i>a key</i>
Experiencer	Entity experiencing something	<i>John</i> heard the noise
Source	Origin of movement	She came <i>from Delhi</i>

Goal	Destination	She went <i>to Mumbai</i>
Location	Place of action	The book is <i>on the table</i>
Time	When action occurs	He arrived <i>at noon</i>
Beneficiary	Who benefits	She cooked for <i>him</i>

Frame Semantics (Fillmore's later work):

- Meaning is understood relative to conceptual frames (background knowledge structures).
- Led to FrameNet — a large lexical database.

Significance in NLP:

- Useful for semantic role labeling (SRL).
- Helps in information extraction, question answering, machine translation.
- More semantically rich than pure syntactic approaches.

9. Grammar-Free Analyzers

Grammar-free (or grammar-less) analyzers do not rely on explicit grammar rules to analyze language. Instead they use statistical, connectionist, or data-driven methods.

Types:

A. Statistical/Corpus-Based Methods

- Use large text corpora to learn patterns.
- **n-gram models:** Predict next word based on previous n-1 words.
- **HMMs:** Used for POS tagging and speech recognition.

B. Case-Based Reasoning

- Analyzes new sentences by analogy to previously seen examples.

C. Connectionist (Neural) Models

- **Recurrent Neural Networks (RNN):** Process sequential text.
- **LSTM/GRU:** Handle long-range dependencies.
- **Transformers (BERT, GPT):** State-of-the-art; use attention mechanisms.

D. Memory-Based Learning

- Stores all training examples and classifies new input by similarity.

E. Semantic Pattern Matching

- Use keywords, patterns, and templates to extract meaning without full parsing.
- Example: ELIZA used pattern-matching with templates.

Advantages:

- No need to hand-craft grammar rules.
- Can handle informal, noisy, or ungrammatical text.
- Scale to large vocabularies.

Disadvantages:

- May lack linguistic interpretability.
 - Require large amounts of data.
-

10. Sentence Generation and Translation

Sentence Generation (Natural Language Generation — NLG)

NLG is the process of automatically producing natural language text from non-linguistic data or internal representations.

NLG Pipeline (Reiter & Dale, 2000):

1. **Content Determination:** What information to include.
2. **Document Structuring:** How to organize information.
3. **Aggregation:** Combining information into coherent units.
4. **Lexicalization:** Choosing appropriate words.
5. **Referring Expression Generation:** Deciding how to refer to entities.
6. **Realization:** Converting abstract representation into actual text (applying grammar and morphology).

Applications: Weather reports, medical summaries, chatbots, data-to-text systems.

Machine Translation (MT)

MT is the task of automatically translating text from one language (source) to another (target).

Approaches:

Approach	Method	Example Systems
Rule-Based MT	Linguistic rules, bilingual dictionaries	SYSTRAN
Statistical MT (SMT)	Phrase-based models from parallel corpora	Moses
Example-Based MT	Translation by analogy	—
Neural MT (NMT)	Encoder-Decoder with attention	Google Translate, DeepL

Neural MT Architecture:

- **Encoder:** Reads source sentence → context vector.
- **Decoder:** Generates target sentence from context.
- **Attention Mechanism:** Allows decoder to focus on relevant source words.

Evaluation Metrics:

- **BLEU Score** (Bilingual Evaluation Understudy): Measures overlap between machine and human translation.
- **METEOR, ROUGE:** Other evaluation metrics.

Challenges: Morphological complexity, word order differences, idiomatic expressions, low-resource languages.

11. Introduction to Pattern Recognition

Pattern Recognition is the process of classifying input data into categories based on key features, usually using statistical or machine learning methods.

Definition: A pattern is a regularized entity that can be described by measurable features. Pattern recognition assigns an unknown input to one of several predefined classes.

Components:

1. **Sensing:** Acquiring input (image, speech, text).
2. **Preprocessing:** Noise removal, normalization.
3. **Feature Extraction:** Identifying relevant features.
4. **Classification:** Assigning input to a class.
5. **Post-processing:** Refining results.

Types of Pattern Recognition:

- **Statistical:** Uses probability distributions (Bayes classifier, k-NN).

- **Syntactic/Structural:** Uses grammars and formal languages to describe patterns.
- **Neural Network-based:** Deep learning classifiers.
- **Template Matching:** Comparing input with stored templates.

Applications: OCR, face recognition, fingerprint identification, medical imaging, speech recognition.

12. Structured Description

Structured descriptions represent patterns using formal structures such as graphs, trees, or strings, rather than just feature vectors.

Key Idea: Complex patterns can be described by the arrangement and relationships of simpler sub-patterns (primitives).

Primitives: Basic elements that make up a pattern. Example: line segments in character recognition, phonemes in speech.

Structural Methods:

1. **String Grammars:** Patterns described as strings over primitives; analyzed using formal grammars (e.g., CFG, regular grammar).
2. **Tree Grammars:** Patterns described as trees; used for 2D patterns.
3. **Graph Grammars:** Most general; pattern described as a labeled graph.
4. **Shape Grammar:** Describes visual shapes via grammar rules.

Example (Character Recognition):

- Primitive: line segments (vertical, horizontal, diagonal, arc).
- "A" = two diagonal lines + one horizontal line.

Advantages:

- Captures spatial/relational structure.
- Interpretable and intuitive.
- Good for hierarchically structured patterns.

Disadvantages:

- Sensitive to noise and distortion.
 - Extraction of primitives is non-trivial.
-

13. Symbolic Description

Symbolic description uses high-level, symbolic representations (logic, frames, semantic networks) to describe objects and patterns.

Key Characteristics:

- **Symbolic:** Uses symbols, predicates, and relations rather than numbers.
- **Declarative:** Describes *what* is known, not *how* to compute.
- **Human-readable:** Representations are interpretable.

Common Formalisms:

A. Predicate Logic

- Describe objects and relations: `circle(X), inside(X, Y), larger(Y, Z)`

B. Semantic Networks

- Graph-based: Nodes = concepts, Arcs = relations (IS-A, PART-OF).
- Example: `Bird IS-A Animal`, `Eagle IS-A Bird`, `Eagle HAS Wings`.

C. Frames (Minsky, 1975)

- Slot-filler structures representing stereotyped situations.

```
Frame: CHAIR
  Legs: 4
  Material: wood/plastic
  Function: sitting
  Has-back: yes
```

D. Production Rules

- IF condition THEN action (used in expert systems for pattern-based reasoning).

Applications:

- Object recognition in images.
- Scene understanding.
- Knowledge representation in AI.

14. Machine Perception

Machine Perception refers to the ability of machines to interpret and make sense of sensory data from the world (vision, sound, touch) in a manner similar to human perception.

Types of Machine Perception:

Type	Input	Application
Computer Vision	Images, video	Face recognition, object detection
Speech Recognition	Audio	Voice assistants, transcription
Natural Language Understanding	Text	Chatbots, QA systems
Tactile Perception	Touch sensors	Robotics
Sensor Fusion	Multiple modalities	Self-driving cars

Challenges:

- High-dimensional input data.
- Real-world variability (lighting, noise, occlusion).
- Ambiguity in interpretation.

Approaches:

1. **Signal Processing:** Low-level feature extraction.
2. **Statistical Learning:** Pattern recognition from data.
3. **Deep Learning:** CNNs for vision, RNNs/Transformers for speech and language.

Levels of Perception:

- **Low Level:** Edge detection, noise filtering.
- **Mid Level:** Object segmentation, feature grouping.
- **High Level:** Scene understanding, semantic interpretation.

15. Object Identification

Object identification (or object recognition) is the task of detecting and classifying objects within an image or scene.

Steps in Object Identification:

1. **Image Acquisition:** Capture image via camera/sensor.
2. **Preprocessing:** Noise removal, contrast enhancement.
3. **Segmentation:** Separate objects from background.
4. **Feature Extraction:** Extract shape, color, texture, SIFT, HOG features.
5. **Classification:** Match features to known object classes.

Classical Approaches:

- **Template Matching:** Compare image regions with stored templates.
- **Generalized Hough Transform:** Detect shapes.
- **SIFT/SURF Features:** Scale-invariant feature extraction.

Deep Learning Approaches:

- **CNN (Convolutional Neural Network):** Automatic feature learning for classification.
- **YOLO (You Only Look Once):** Real-time object detection.
- **R-CNN, Faster R-CNN:** Region-based object detection.
- **SSD (Single Shot Detector):** Fast detection in one pass.

Challenges:

- Viewpoint variation (same object, different angles).
- Occlusion (partially hidden objects).
- Intra-class variation.
- Scale and illumination changes.

Evaluation Metrics:

- **Precision, Recall, mAP (mean Average Precision)** for detection.
 - **Top-1 / Top-5 accuracy** for classification (e.g., ImageNet benchmark).
-

16. Speech Recognition

Speech Recognition (Automatic Speech Recognition — ASR) is the task of converting spoken language into text.

Pipeline:

1. **Signal Acquisition:** Microphone captures audio.
2. **Preprocessing:** Noise filtering, pre-emphasis, framing.
3. **Feature Extraction:** MFCC (Mel-Frequency Cepstral Coefficients) — most common.
4. **Acoustic Model:** Maps features to phonemes (HMM, DNN).
5. **Language Model:** Provides word sequence probabilities (n-gram, neural LM).
6. **Decoder:** Combines acoustic + language model to output most likely word sequence (Viterbi algorithm / Beam search).

Key Concepts:

Concept	Description
Phoneme	Smallest unit of sound
MFCC	Audio features representing spectral shape
HMM	Models temporal variability of speech
Acoustic Model	$P(\text{audio} \mid \text{words})$
Language Model	$P(\text{word sequence})$
Beam Search	Efficient decoder for best hypothesis

Types of ASR Systems:

- **Speaker-Dependent vs. Speaker-Independent**
- **Isolated word vs. Continuous speech**
- **Small vocabulary vs. Large vocabulary**

Modern Deep Learning ASR:

- **Deep Neural Network-HMM (DNN-HMM):** Replaces GMM with DNN.
- **End-to-End Models:** CTC (Connectionist Temporal Classification), Seq2Seq with attention.
- **Transformer-based:** Whisper (OpenAI), wav2vec 2.0 (Meta).

Challenges:

- Accents and dialects.
- Homophones: "their" vs. "there".
- Noise and overlapping speech.
- Real-time processing requirements.

Evaluation Metric:

- **WER (Word Error Rate)** = $(\text{Substitutions} + \text{Deletions} + \text{Insertions}) / \text{Total Words} \times 100$

Quick Revision: 5-Mark Question Answers Summary

Topic	Key Points to Cover
Natural Language	Levels of analysis, characteristics, challenges
NLP Approaches	Rule-based, Statistical ML, Deep Learning

NLP Approaches	Rule-based, Statistical, ML, Deep Learning
Parsing	Top-down, Bottom-up, CYK, Earley
CFG	Definition (V,T,P,S), rules, parse trees, ambiguity
Transformational Grammar	Deep/surface structure, Chomsky, transformations
Transition Nets	Nodes/arcs, RTN, limitations
ATN	Registers, conditions, actions, arc types, Woods
Fillmore's Case Grammar	Agent, Patient, Instrument, cases table
Grammar-Free Analyzers	Statistical, neural, advantages/disadvantages
Sentence Generation	NLG pipeline 6 steps
Machine Translation	Rule-based, SMT, NMT, BLEU
Pattern Recognition	Components, types, applications
Structured Description	Primitives, string/tree/graph grammars
Symbolic Description	Logic, semantic nets, frames
Machine Perception	Types, levels, challenges
Object Identification	Steps, classical, deep learning, metrics
Speech Recognition	Pipeline, MFCC, HMM, WER, modern models